



Rebuild a live client site as a Next.js and Supabase site

How an existing client website becomes a new Next.js site built from the HGM template, matched section by section against the original. If you remember one thing: **the live site is the reference, not your memory** — every section is judged against a saved capture of the original, never against what looks about right on screen.

SOP ID	HGM-SOP-WEB-004	EFFECTIVE	2026-09-23
VERSION	0.5	NEXT REVIEW	2027-03-23
OWNER	Web & Content Specialist (Kyle Zinger)	TRIGGER	Every new AI website build assigned in Asana
APPROVED BY	Operations Manager (Gillian)	DURATION	3 to 5 working days (provisional)

01 PURPOSE AND SCOPE

Purpose

Turns an existing client website into a new Next.js site built from the HGM AI website template, hosted on Netlify, with Supabase as the database and the client's property management system as the booking API. The finished site is a faithful copy of the original on its `netlify.app` address, ready for pre-launch checks.

In scope

Making the repository, connecting Netlify, collecting keys, gathering reference material, writing and executing the plan, matching every page and section to the original, building the forms and pop-up, and recording what the build cost in tokens.

Out of scope

Pricing and payment paths, SEO and accessibility audits, Lighthouse, BrowserStack, the test booking, legal pages and the DNS cutover — all of those belong to the pre-launch SOP. Analytics and Search Console belong to WEB-003, after launch.

This SOP covers the case where a **live website already exists** and is being reproduced. Building a site from a Figma design with no live original is a different job with a different reference: see section 09.

Five standing rules

These hold on every build. Everything else in this document is procedure; these are not negotiable.

1

The live site is the reference

Every section is judged against a saved capture of the original in Resources/, not against what looks right on screen. If there is no capture, make one before you build the section.

2

Never touch pricing or payment

The booking flow's money path — prices, deposits, Stripe, coupons — is not built or altered in this SOP. Work up to the property page and stop. The money path is handled in the pre-launch SOP.

3

Secret keys go in two places and nowhere else

.env.local on your machine and the client's 1Password vault. Never in a commit, never in a prompt, never in a screenshot. The Supabase service role key bypasses row level security — treat it as a password.

4

Nothing gets a custom domain here

The site lives on its *.netlify.app address for the whole of this SOP. The DNS cutover happens at launch, in the pre-launch SOP, after the client has approved the site.

5

Record what it cost

Every working session appends its token count and dollar cost to TOKENS.md in the repository root. Operations needs a real cost per build, and nobody can reconstruct it afterwards.

ROLE	RESPONSIBILITY
Web team member running the build	Performs tasks 1.1 to 8.4. Anyone on the web team can run this SOP.
Web & Content Specialist (Kyle Zinger)	Owns this SOP and keeps it current. First point of contact when the procedure and reality disagree.
Template owner (Leshan Patterson)	Owns the two template repositories. Escalation point when the template itself is wrong, stale or missing a file.
Operations Manager (Gillian)	Approves this SOP. Approves scope changes that add days to a build.
Client websites space, Google Chat	Where the build is posted for the team to review for mistakes and issues before it moves to pre-launch.
Client	Owns the Supabase project and the property management system account. Supplies keys and reviews the finished site.

1 Access to the hgmwebteam GitHub organisation

Enough to create a repository from a private template. Ask the template owner if **Use this template** is not offered.

2 The client's Netlify login

Netlify is the client's own account, not HGM's. The credentials are on the client's record in hgmportal.com.

3 Access to the client's GoHighLevel sub-account

The site's forms are built there and embedded here. You need the sub-account, not the agency view.

4 A login for hgmportal.com

The portal holds the client record: which property management system they are on, and the keys HGM already has.

5 1Password, with the client's vault

Client logins and the site's `.env.local` entry live here. See HGM-SOP-WEB-001.

6 VS Code with Claude Code signed in

The build is driven from Claude Code inside VS Code. Node and npm installed — HGM-SOP-WEB-002 task 1.1 lists the four tools.

7 The URL of the live site

The site being reproduced. It must still be reachable; once DNS moves at launch it is gone.

8 A browser with developer tools

The original site's stylesheets and computed styles are where the design tokens come from. Chrome or Firefox, with the Elements and Network panels.

9 The Asana task

The build request. It carries the client, the deadline and anything the account manager has already agreed.

Where are you working?

VS Code

The whole build. Clone, edit, run the site locally, and drive Claude Code. This is the normal route.

Terminal

Install, run and build commands. Everything here also works from the VS Code terminal.

Claude Desktop app

Prompts only. Useful for reading a plan or drafting a correction prompt away from the machine.

04 DEFINITIONS

Property management system (PMS)

The booking platform the client's properties live in — Guesty or Hostaway. It supplies listings, availability and rates through its API. Which one the client uses decides which template you start from, and it is recorded on the client's record in hgmportal.com.

Template repository

A GitHub repository marked **Private template**. Pressing **Use this template** makes a new repository containing a copy of its files, with no shared history. HGM has two: ai-website-template-guesty and ai-website-template-hostaway.

The three places the site exists

Local — the copy running on your machine at localhost, the only one you edit.
Preview — a Netlify build of a branch or pull request, on a temporary address.
Production — the build of main, on the site's netlify.app address. A change is only in the place you put it.

Resources folder

Resources/ in the repository root. Holds the saved HTML and screen captures of the original site and the design exports. It is the evidence the build is checked against, and it stays in the repository.

PLAN.md

The build plan, shipped in the template and rewritten for each client. It names the template, the client, the property management system, the host, the database and the site being reproduced, and lists the work section by section.

Design tokens

The small set of repeated values a site's look is built from — its colours, font families and weights, size scale, spacing steps, corner radii and breakpoints. On the original they live in its stylesheets; here they are collected once into Resources/VARIABLES.md and the build is held to them.

What are you trying to do?

Pick the phase that matches your situation and jump straight to its tasks.

PHASE 1

Start the site's repository

Tasks 1.1–1.4 · once, at the start of a build

PHASE 2

Connect the repository to Netlify

Tasks 2.1–2.3 · before any code is written

PHASE 3

Open the site on your machine

Tasks 3.1–3.4 · first time on this computer or site

PHASE 4

Collect the keys

Tasks 4.1–4.7 · as much as you can get today

PHASE 5

Build the Resources folder

Tasks 5.1–5.7 · before you write a line of the plan

PHASE 6

Write the plan

Tasks 6.1–6.4 · get a plan, not code

PHASE 7

Build it and match it

Tasks 7.1–7.10 · the long part of the job

PHASE 8

Close out and hand on

Tasks 8.1–8.4 · cost, review, next SOP

Phase 1 — Start the site's repository

Pick the right template for the client's booking platform and make the site's own repository from it. Do this once, at the start of a build.

1.1 Find out which booking platform the client uses

Open **hgmportal.com**, find the client's record, and read which property management system they are on.

Do not guess from the old site

A WordPress site can front any booking platform, and some front more than one. The portal record is the answer. If the portal has no record, ask the account manager named on the Asana task before you start.

Image to be added

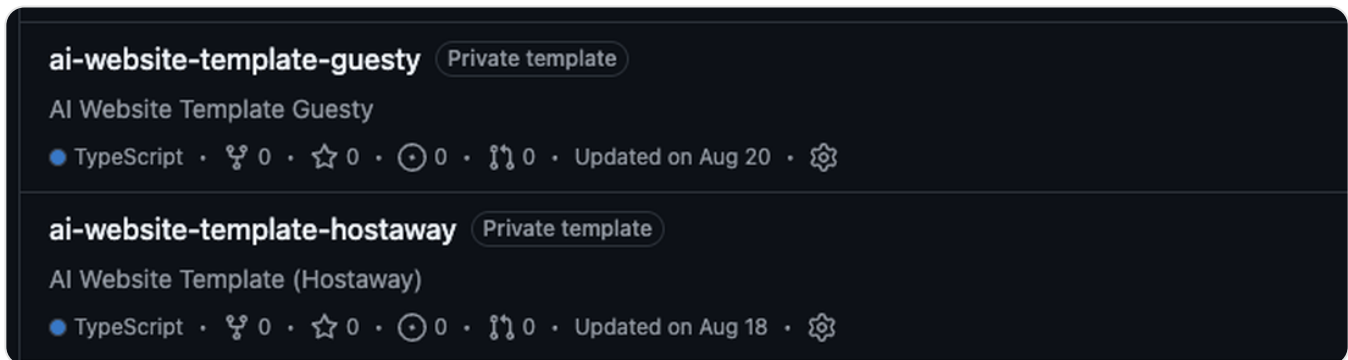
(image to be added) — The client's record on hgmportal.com, cropped to the property management system field. Ring the PMS field.

Expected result

You know the client is on Guesty or on Hostaway. Every later choice — template, plan, API keys — follows from this one.

1.2 Open the matching template repository

In GitHub, open the hgmwebteam organisation and open **ai-website-template-guesty** or **ai-website-template-hostaway**, whichever matches the answer from task 1.1.



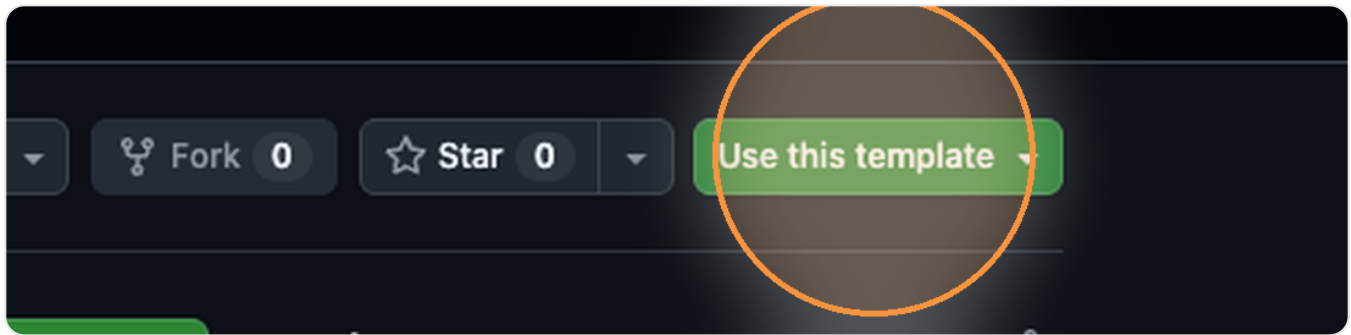
The two site templates in the hgmwebteam organisation. Pick the one matching the client's booking platform.

Expected result

The repository page shows the **Private template** badge beside its name.

1.3 Create the site's repository from the template

Click **Use this template**, then **Create a new repository**. Set the owner to **hgmwebteam**, name it after the client in lower case with hyphens — for example **ridge-and-falls** — and set it to **Private**.



Use this template, at the top right of the template repository.

Why a template and not a fork

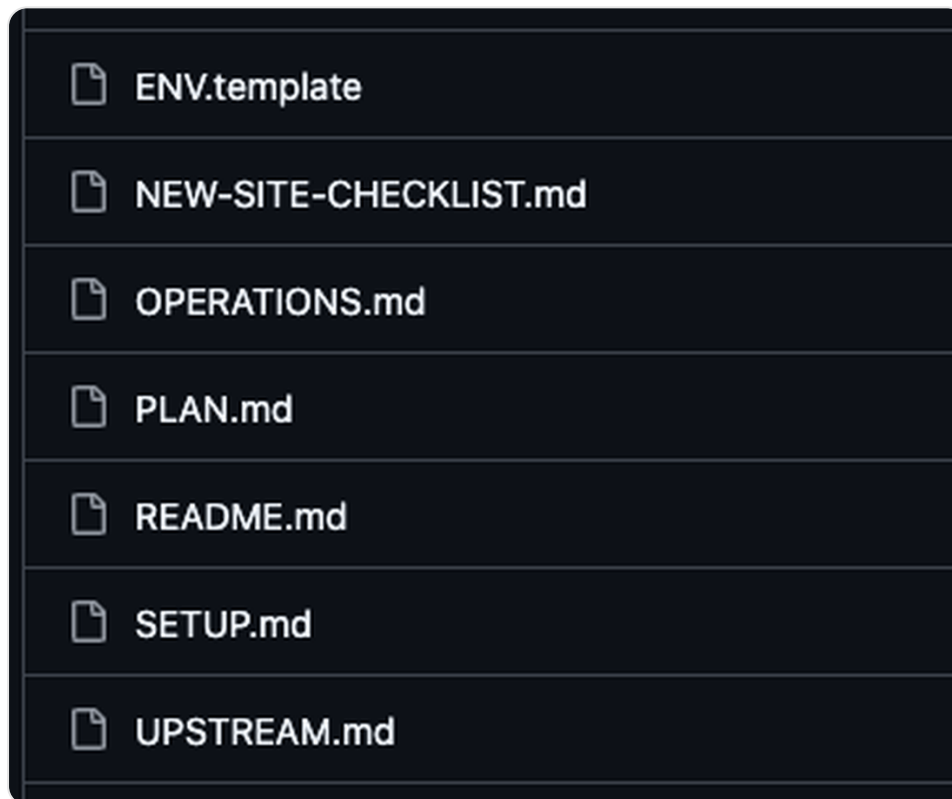
A fork keeps a link back to the template and carries its history. A template copy starts clean, which is what a client site wants — the template's own history is not this client's history.

Expected result

A new repository exists at `github.com/hgmwebteam/<client-name>` with the template's files and no commit history from the template.

1.4 Read the template's own documentation before touching anything

In the new repository, read `README.md`, `SETUP.md`, `NEW-SITE-CHECKLIST.md` and `UPSTREAM.md`. They describe this version of the template, which moves faster than this SOP does.



The template's documentation files, beside PLAN.md and ENV.template in the repository root.

⊗ **Where the template and this SOP disagree, the template wins**

The template is the running code. If SETUP.md contradicts a task here, follow SETUP.md, finish the build, then tell the Web & Content Specialist so this document gets fixed rather than quietly ignored.

👁 **Expected result**

You know what the template already ships — booking engine, admin dashboard, checkout, email — so you do not rebuild something that is already there.

Phase 2 — Connect the repository to Netlify

Get HGM onto the client's Netlify account, then give the site a real address before any code is written — the address is itself a value the build needs.

2.1 Sign in to the client's Netlify account and add HGM to it

Sign in to Netlify with the client's credentials from their record on **hgmportal.com**. Go to **Team settings** → **Members**, invite the HGM web team Google account — the shared address on the same portal record — and give it the **Developer** role.

Developer, not Owner

Netlify's own guidance is to grant the least access that does the job. **Developer** covers everything this SOP needs. **Owner** additionally carries billing and the ability to delete the team, which is not HGM's to hold on a client account. If the client's plan only offers Owner, Collaborator and Billing Admin, choose **Collaborator**.

The account is the client's, permanently

The client pays for and owns the Netlify team. HGM is a member of it. That is what lets the relationship end without the site having to move.

Image to be added

(image to be added) — Netlify **Team settings** → **Members** with the invite dialog open and the role list showing. Ring the **Developer** option.

Expected result

The HGM web team account can reach the team, create projects, set environment variables and deploy, without being able to change billing or delete the team.

2.2 Import the repository into Netlify

In Netlify, click **Add new project**, then **Import an existing project**, then **GitHub**, then choose hgmwebteam/<client-name>. Accept the build settings the template supplies and click **Deploy**.

Connect the repository, do not create the site from the command line

A site created with the Netlify CLI has no Git connection, so nothing rebuilds when anyone pushes. Importing the repository is what makes every later push publish.

Image to be added

(image to be added) — Netlify **Import an existing project** → **GitHub**, with the repository list showing. Ring the repository row.

Expected result

A Netlify project exists and its first build starts. That build will fail, because no environment variables are set yet. That is expected and is fixed in task 4.6.

2.3 Record the site's address

Copy the project's *.netlify.app address from the Netlify project overview and paste it into the Asana task as a comment.

No custom domain in this SOP

Pointing the client's real domain at this project takes their current site down. The DNS cutover happens at launch, after the client has approved the build — see the pre-launch SOP in section 09.

Expected result

The team can reach the build without asking you, and you have the value task 4.5 needs.

Phase 3 — Open the site on your machine

Get your own copy running locally. If this is your first time on this computer, do the machine setup in HGM-SOP-WEB-002 first.

3.1 Clone the repository into VS Code

In a new VS Code window, open the Explorer and click **Clone Repository**, then paste the repository URL from GitHub's **Code** button.

The long version lives in WEB-002

HGM-SOP-WEB-002 tasks 1.1 to 1.7 cover installing the four tools, cloning and running `npm install`, with a screenshot on every click. Follow it the first time; come back here once the repository is open.

 OR CLONE FROM THE
TERMINAL

```
git clone https://github.com/hgmwebteam/<client-name>.git
cd <client-name>
```

Expected result

VS Code asks where to save it, then asks whether to open it. Open it.

3.2 Install the packages

Open a terminal in the project folder and run `npm install`.

 IN THE PROJECT
FOLDER

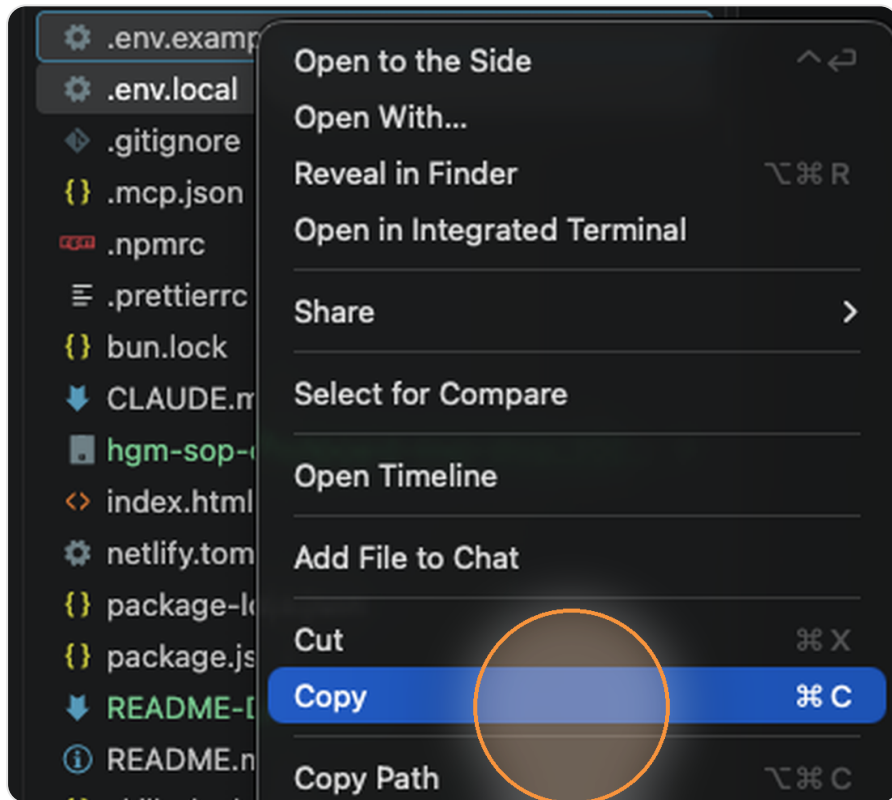
```
npm install
```

Expected result

The command finishes without an error and a `node_modules` folder appears. It takes a few minutes the first time.

3.3 Make your own environment file

In the Explorer, right-click the template's environment example file in the repository root — `ENV.template` in the current templates — choose **Copy**, then **Paste**, and rename the copy to `.env.local`.



Right-click the environment example file and choose Copy. The same menu carries Copy Relative Path, used later when a prompt needs an exact file path.

⊗ Do not put NETLIFY_SITE_ID in .env.local

Setting it locally flips the site into its deployed-Netlify behaviour while you are on localhost: images start requesting Netlify's image CDN and 404, and the local booking store points at the deployed site's storage. Leave it unset locally and set it on Netlify only.

👁 Expected result

.env.local sits in the repository root and is greyed out in the Explorer, because .gitignore excludes it. If it is not greyed out, stop and check .gitignore before you put a key in it.

3.4 Start the cost file

Create TOKENS.md in the repository root with one heading and a table — date, session, approximate tokens, cost in Canadian dollars — and commit it.

CREATE AND COMMIT THE
FILE

```
printf '# Token spend\n\n| Date | Session | Approx. tokens | Cost (CAD) |\n|---|---|---|---|\n' >  
TOKENS.md  
git add TOKENS.md && git commit -m "chore: start token spend log"
```

Expected result

Every later session has somewhere to record what it cost. Task 8.1 fills in the first row.

Phase 4 — Collect the keys

Fill in as much of the environment file as you can get today, set the same values on Netlify, and write the result back to 1Password.

4.1 Take the keys HGM already holds from the portal

Open **hgmportal.com**, open the client's record, and copy every key it holds for this client into `.env.local`, matching each one to the variable name in `ENV.template`.

Some values are the same for every client

A small number of platform credentials are held once at the agency and reused — the portal record is where you find out which. Never copy one from another client's `.env.local`; take it from the portal.

Image to be added

(image to be added) — The keys section of the client's record on hgmportal.com, with every value redacted before you send it.

Expected result

The variables HGM can supply without the client are filled in.

4.2 Get into the client's Supabase project and add yourself to it

Sign in to Supabase with the client's credentials from 1Password, open their project, then invite your own HGM account as a member under **Project settings** → **Members**.

The client owns this project

The Supabase project is created in the client's own account, not HGM's. Adding yourself as a member is what keeps the audit trail honest and survives a password change on their side.

Image to be added

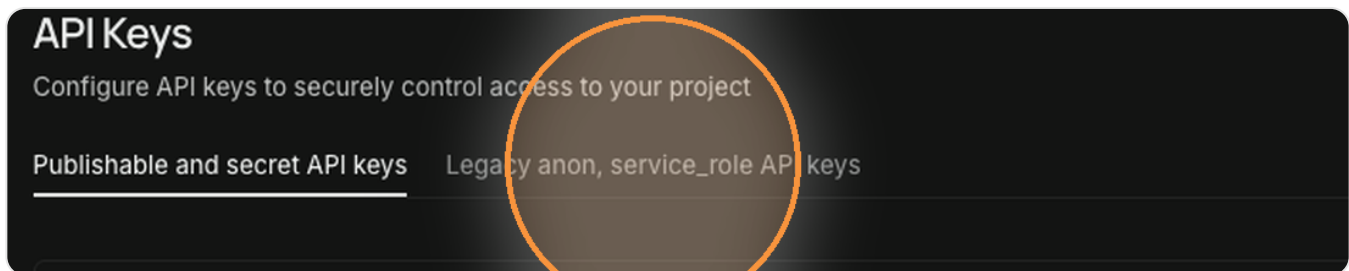
(image to be added) — Supabase **Project settings** → **Members** with the invite dialog open. Ring the **Invite** button.

Expected result

You can reach the project signed in as yourself, and you stop using the client's login from this point on.

4.3 Open the project's API keys

In the Supabase project, go to **Project settings** → **API Keys**, then click the **Legacy anon, service_role API keys** tab.



The API Keys page. Click the Legacy anon, service_role API keys tab — the first tab shows the newer key pair, which the template does not use.

Expected result

The tab shows an anon key and a service_role key. The template expects these two, not the newer publishable and secret pair on the first tab.

4.4 Copy the two Supabase values into the environment file

Copy the project URL and the anon key into their NEXT_PUBLIC_ variables, and the service_role key into its own variable, exactly as ENV.template names them.

The service role key is a master key

It ignores row level security and can read and write every row in the client's database. It must never sit in a variable whose name starts NEXT_PUBLIC_, never be committed, and never appear in a screenshot. If one is ever exposed, rotate it in Supabase the same day and tell the template owner.

Expected result

Three Supabase values are set and nothing has been pasted into a chat window.

4.5 Set the site's own address

Set `NEXT_PUBLIC_SITE_URL` to the `*.netlify.app` address you recorded in task 2.2, with no trailing slash.

Expected result

The variable is set. Leaving it empty makes the site throw during the build rather than falling back to a default.

4.6 Put the same variables on Netlify and rebuild

In the Netlify project, open **Site configuration** → **Environment variables**, add each variable from `.env.local` by hand, then trigger a new deploy.

Add them by hand, one at a time

Bulk-importing `.env.local` uploads everything in the file, including anything personal to your own machine, and arms every scheduled function the template ships. Type in the variables the site needs and no others.

Image to be added

(image to be added) — Netlify **Site configuration** → **Environment variables**, mid-way through adding one, values redacted. Ring the **Add a variable** button.

Expected result

The deploy succeeds and the `netlify.app` address serves the template's home page.

4.7 Write the environment file back to 1Password, and note what is missing

Save the current `.env.local` into the client's 1Password vault following HGM-SOP-WEB-001, then comment on the Asana task listing every variable still empty and who has to supply it.

Deferring a key is normal

Fill in everything you can get in the moment and defer the rest to launch. What is not normal is nobody knowing a key is missing until the day the site is meant to go live.

Expected result

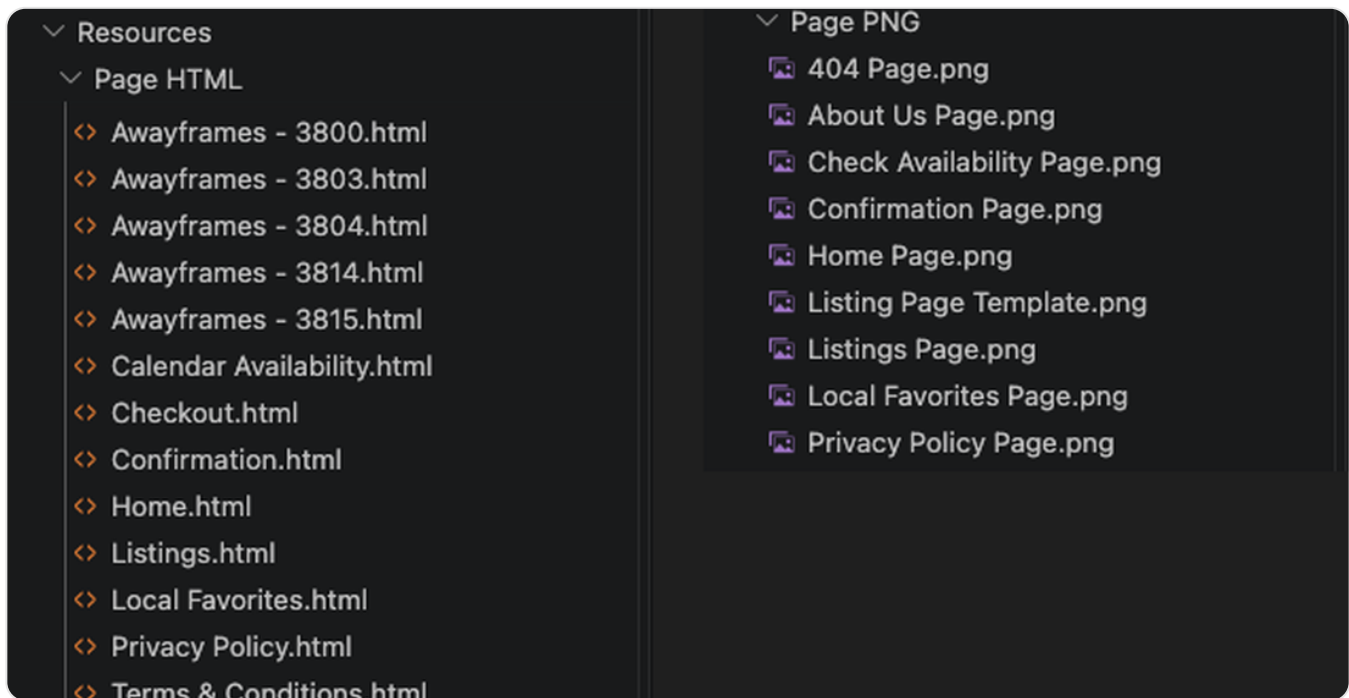
Someone other than you could pick the build up tomorrow. The missing keys — typically Stripe, Resend and the booking platform's live credentials — are somebody's named job rather than a surprise at launch.

Phase 5 — Build the Resources folder

Gather the evidence the build gets checked against. Doing this properly is what makes phase 7 finite.

5.1 Create the folder structure

In the repository root, create Resources/ containing Page HTML/, Page PNG/, Design System/ and CSS/.



Resources/Page HTML and Resources/Page PNG on a finished build — one file per page of the original site, named for the page.

Expected result

Four empty folders, committed to the repository. They stay there — the evidence lives with the code.

5.2 Save the HTML of every page on the original site

For each page in the original site's sitemap, save the rendered page source into `Resources/Page HTML/`, named for the page — `Home.html`, `Listings.html`, `Checkout.html` and so on.



ONE PAGE, FROM THE
TERMINAL

```
curl -sL "https://<live-site>/<page>" -o "Resources/Page HTML/<Page>.html"
```



Expected result

One HTML file per page. This is the copy that carries exact wording, field names and widget settings, which a screenshot cannot.

5.3 Capture every page as a full-page image

Capture each page of the original site top to bottom at 2× pixel density and save it into `Resources/Page PNG/`, named for the page.



Keep anything you attach to a prompt under 2000px wide

Claude handles an attached image poorly once it is wider than about 2000 pixels — it reads the picture but stops seeing the detail you are asking about. Capture at 2× for the archive, then resize the copy you attach so its longest edge is 2000 pixels or less.



Expected result

One image per page, showing the whole page rather than the visible window.

5.4 Capture each section, the header and the footer separately

Capture every distinct section of the original — hero, search widget, listing grid, reviews, calls to action — plus the header and footer, at 2×, and save them into `Resources/Design System/`.



Expected result

A section-level library. Phase 7 attaches one of these per correction prompt, and a section capture is readable where a full-page capture is not.

5.5 Save the original site's stylesheets

Open the original site with the browser's **Network** panel filtered to CSS, reload, and save every stylesheet the page loads into Resources/CSS/ under its own filename.

i Where the values actually live

On a WordPress and Elementor site the global kit stylesheet holds the whole palette as literal hex values, plus the type scale. On a modern build it is usually a block of CSS custom properties near the top of the main stylesheet. Either way it is machine-readable, which a capture is not.

i Image to be added

(image to be added) — The browser's **Network** panel filtered to CSS on the original site, showing the stylesheet list. Ring the filter control.

👁 Expected result

The site's real design values are now on disk as text. This is the exact source — a screenshot is an approximation of it.

5.6 Build the variables file from the stylesheets

In Claude Code, run the prompt below against Resources/CSS/. Save the result as Resources/VARIABLES.md.

🔗 BUILD THE VARIABLES FILE

Read every stylesheet in Resources/CSS/ and write Resources/VARIABLES.md.

List the design tokens this site actually uses: colour ramps and semantic colours as hex, font families, font weights, the font-size scale, line heights, the spacing scale, border radii, shadows and breakpoints.

For every value, give the file and the declaration it came from. Where one role carries more than one value across the site, list them all and say which pages use which.

Do not infer a value you cannot point at in the CSS. List it under "Unresolved" instead, with what you were looking for.

Build it from the CSS, not from the captures

Asking for tokens from screenshots produces plausible hexes that are one or two shades out, and nobody catches it until the client does. The stylesheet carries the real numbers. Use the captures to check the file, not to write it.

Expected result

A Markdown file listing the site's colours, fonts, sizes, spacing, radii and breakpoints, each traceable to the declaration it came from, and each value you cannot trace listed as unresolved rather than filled in.

5.7 Check the variables against the rendered page, then commit

Open three pages of the original in the browser. On each, use **Inspect** → **Computed** on the main heading, a paragraph of body text and the primary button, and compare the font, size, weight, colour and radius against `VARIABLES.md`. Correct the file where they differ, then commit the whole `Resources/` folder.

Where the file and the page disagree, the page wins

A stylesheet can carry declarations that something later overrides, so the computed value is the truth. Fix `VARIABLES.md` to match what renders, and note the override in the file so nobody re-introduces the dead value.

Capture the states a screenshot misses

While you are in the browser, capture the hover states, the open mobile menu, a form with a validation error showing, and any empty or sold-out state. Save them into `Resources/Design System/`. These are the details phase 7 is most likely to get wrong, because nothing in a static capture shows them.

Image to be added

(image to be added) — Devtools with **Computed** open on a heading, with font, size and colour visible.

Expected result

Every value you spot-checked matches what the browser actually renders, and everything the build will be judged against is pushed rather than sitting in one person's Downloads folder.

Phase 6 — Write the plan

Rework the template's plan for this client, read it yourself, and only then let anything be built.

6.1 Replace the template's placeholder client everywhere

Open `PLAN.md` and run find and replace across the file: the placeholder client name the template ships with, replaced by this client's name.

This is the most common mistake in the whole build

A leftover client name in the plan gets built into the site — wrong phone number, wrong branding, wrong copy — and it is found weeks later by the client. Check the whole file, not just the headings.

Expected result

No other client's name survives anywhere in `PLAN.md`. Search the file for the old name afterwards and confirm zero results.

6.2 Replace the property management system throughout

In the same file, replace every mention of the template's booking platform with the one this client uses, if they differ.

Expected result

`PLAN.md` names one booking platform and it is the client's.

6.3 Ask Claude to rewrite the plan against this project

In Claude Code, with the repository open, paste the prompt below with the four values filled in.

REWORK THE PLAN

FILL IN THE FOUR ANGLE-BRACKET VALUES

Go through `plan.md` and update all the sections to reflect creating the website using the template which we defined at the top of the file. The website will be created and will be hosted on Netlify. The database will be Supabase, and the API will be `<Guesty or Hostaway>`. We are recreating the `<live site URL>` website, which was built using `<WordPress/Elementor, or whatever the original was built in>`, as a Next.js site. The plan should highlight in detail, with the same sections that are already in place, reproducing the website using the current plan. Update what is necessary.

Point the plan at the evidence

Add a line naming `Resources/` as the reference set — the saved HTML, the captures and `VARIABLES.md` — so the plan is written against what is on disk rather than against the live site the model cannot see.

Expected result

`PLAN.md` now describes this client's build section by section, against this template, with the original site named as the target.

6.4 Read the plan yourself before executing it

Read `PLAN.md` end to end. Check the goal names the right site, the success conditions are ones you could test, every page in the original's sitemap appears, and any page deliberately left out is named as such.

```
PLAN.md > # PLAN — Rebuild awayframes.com as a Next.js site on the Hostaway template
1 # PLAN — Rebuild awayframes.com as a Next.js site on the Hostaway template
2
3 Dictionary or definition: this template means the files that already exist in this repository — the
4 AI Website Template — Hostaway PMS Integration (see [README.md](README.md)), a production-grade
5 direct-booking site cloned from the Taberg Falls production site at commit `7238f0a`, carrying
6 placeholder branding ("Lush Hollow") everywhere a new client is meant to substitute their own. This
7 will be a direct booking site that uses the Hostaway API, with every API key held in one file,
8 .env.local — one file, see §1.1 for why that exact name. The website will be hosted on
9 Netlify, and the database will be Supabase. The visual and content target is the existing
10 WordPress/Elementor site at https://awayframes.com.
11
12 Created: 2026-07-31 · Rewritten against the template + live recon: 2026-08-06 ·
13 Verified adversarially: 2026-08-06 · Mode: Sentinel Ultra ·
14 Status: recon complete, foundation not started
15
16 ---
17
18 ## 0. Goal
19
20 Replace the WordPress/Elementor site at https://awayframes.com with the Next.js application in
21 this repository, rebranded from "Lush Hollow" to Awayframes, wired to Awayframes' own Hostaway
22 account, Supabase project, Stripe account and Netlify site.
23
24 Three success conditions, in priority order:
```

The top of a reworked `PLAN.md` — it names the template, the original site, the host, the database, the booking API, and the success conditions in priority order.

Nothing is built until the plan is right

Executing a wrong plan costs a day and a large share of the build's token budget, and the result looks finished enough that the error survives review. Reading the plan takes fifteen minutes.

Expected result

You could explain to Gillian what is being built and what is deliberately not. If you cannot, fix the plan before task 7.1.

Phase 7 — Build it and match it

Execute the plan, then close the gap between your copy and the original, section by section. This is where most of the elapsed time goes.

7.1 Execute the plan

In Claude Code, on a branch off `main`, run the prompt below and let it work through `PLAN.md`.

✦ EXECUTE THE
PLAN

Execute `Plan.md`, ensure there are no mistakes. Everything must be perfect.

👁 Expected result

The site runs on `localhost` with every page in the plan present. It will not yet match the original.

7.2 Close the obvious gap against the original

Run the prompt below with the original site's URL. Use it once, to get from roughly right to close.

✦ MATCH THE
ORIGINAL

DROP THE LAST LINE IF SENTINEL MODES ARE NOT SET UP

The website does not look exactly the same, create a loop and loop until everything looks exactly like `<live site URL>`. There can be no mistakes, everything must be perfect.

Use Sentinel Ultra and Sentinel.

⚠ A loop is a blunt instrument

Looping against a whole site gets you close and then stops improving, while spending tokens on every pass. Run it once, then switch to task 7.3. Looping a second time is the single easiest way to waste a day.

👁 Expected result

Structure, page order and the large layout decisions now follow the original. Fine detail will not, and that is task 7.3's job.

7.3 Correct the site section by section, in one pass

Open the original and your local site side by side. Write one prompt that names every section that is wrong, says what is wrong with each, and attaches the matching capture from Resources/Design System/ plus a capture of your version.



A REAL CORRECTION
PASS

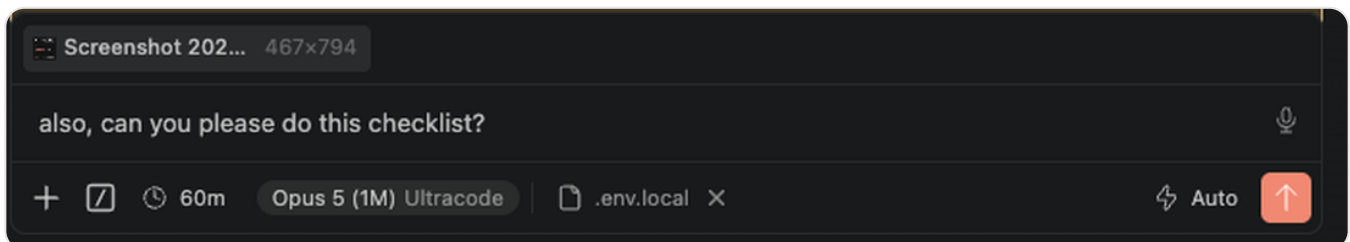
ONE PROMPT, SEVERAL NUMBERED CORRECTIONS

- 1) Edit the listings page's Quick tour section to remove the gallery button on every image and just have each image link to the gallery.
- 2) Can we go three wide here, not four? It is too squished.
- 3) Wrong phone number – it should be the same phone number as the rest of the site.
- 4) Reduce the height of this pop-up card by shrinking the height of the image on the right and removing the large amount of spacing on the left between the form and the title and the bottom of the area. Snug the text up to near the bottom of the form if possible. Good UX/UI please.



One large prompt beats a loop

Addressing every section in a single well-written prompt has consistently produced better results than asking for a loop. Be specific about what is wrong; "make it match" produces a rewrite, and a rewrite breaks the sections that were already right.



The Claude Code prompt box in VS Code — the attached capture sits above the prompt as a chip, and the file in context sits on the toolbar beside the model.



Resize before you attach

Attach nothing wider than 2000 pixels. An oversized capture is read as a picture rather than as evidence, and the correction comes back generic.



Expected result

Each section moves towards the original in one round rather than several. Re-capture your site and repeat until the list is empty.

7.4 Build the form in GoHighLevel

Open the client's GoHighLevel sub-account, go to **Sites** → **Forms**, start from the HGM form template, and adjust the fields to match the form on the original site.

⊗ Check which sub-account you are in

A form built in the wrong sub-account sends one client's enquiries into another client's CRM, and nothing errors — the submission simply lands in the wrong place. Confirm the location before you build, and confirm the Location ID on the client's record in hgmportal.com matches the one you are working in.

ⓘ Labels can differ

HGM's GoHighLevel is white-labelled, so a menu may not read exactly as it does in GoHighLevel's own documentation. Follow the equivalent item rather than looking for the literal words.

ⓘ Image to be added

(image to be added) — GoHighLevel **Sites** → **Forms** with the HGM form template in the list. Ring the template row.

👁 Expected result

A saved form in the client's own sub-account, with the same fields the original site asked for and no extras.

7.5 Copy the form's embed code

In the form builder, click **Integrate form**, choose **Inline** as the embed type, and copy the embed code.

⚠ Take the inline embed even for the pop-up

The inline embed drops into a container you control, which is what the site's own pop-up needs. GoHighLevel's own pop-up embed brings its own overlay and fights the one built in task 7.7.

Image to be added

(image to be added) — The GoHighLevel form builder with the integrate control visible, then the embed dialog with the inline layout selected. Ring the **Integrate form** control.

Expected result

An iframe snippet on your clipboard, pointing at the form you just built.

7.6 Place the forms on the site

Give Claude the embed code and say where each form goes — inline in a named page, or inside the pop-up.

The form is not ours to restyle

Fields, validation and where a submission lands are configured in GoHighLevel, inside the iframe. Everything built here is the container around the embed.

Submit one of each before you move on

Send a test submission through every form on the site and confirm it appears as a contact in the client's GoHighLevel sub-account. A form that renders and does not deliver looks identical to one that works.

Expected result

Each form renders in place, sized to its container, on desktop and on a phone.

7.7 Build the pop-up

Attach the pop-up captures from Resources/Design System/ and ask for the same design at every screen size, with the offer wording taken from the Asana task.



REPLACE THE OFFER WITH THE ONE ON THE ASANA TASK

Build the pop-up to match the attached design. Keep a similar design on all screen sizes but allow different content where the layout needs it. The offer is 10% off, not a fixed dollar amount.

Check the offer against the task, not the design file

Offer wording changes after a design is signed off more often than the design gets updated. A percentage and a dollar amount are not interchangeable, and the wrong one is a discount HGM has published on the client's behalf.

Image to be added

(image to be added) — The finished pop-up on a phone width and on a desktop width, side by side.

Expected result

One pop-up design, correct on a phone and on a desktop, showing the offer the client actually agreed.

7.8 Agree the mobile menu before building it

The original site's mobile menu is rarely in the design and rarely worth copying. Post a capture of the original's mobile menu in the **client websites** space in Google Chat, propose what you intend to build, and get a reply before building it.

POST IN THE CLIENT WEBSITES SPACE

<client> mobile menu — the original uses <describe it>. I am proposing <what you intend to build> instead, because <reason>. Shout before end of day if you would rather I copied the original exactly.

Expected result

A written decision from the designer or the account manager. Build that.

7.9 Add the motion

Ask for the site's animations in one pass, naming the motion skill so the result is consistent rather than improvised per section.

ANIMATIONS

Add an immersive experience by applying the appropriate animations to all the pages – for example elements fading in – using the /emil-design-eng skill.

👁 Expected result

Elements enter and transition consistently across every page, and nothing moves for a visitor who has reduced motion turned on.

7.10 Walk the booking flow as a guest

Start at the search widget on the home page, search, open the listings, and open a property page. Check the dates and guest counts a visitor chooses are carried through every step, and that the URLs are the ones the plan describes.

⊗ Stop at the property page

Prices, deposits, checkout, coupons and anything that creates a charge are not built or altered in this SOP. That work is done in the pre-launch SOP, in Claude Fable, against a test key. A mistake past this line charges a real person the wrong amount.

ℹ Image to be added

(image to be added) — The finished build's search widget with dates and guests filled in, and the property page it leads to.

👁 Expected result

A guest can get from the home page to a property page with their dates intact. Stop there.

Phase 8 — Close out and hand on

Record what the build cost, get it reviewed by the team, and pass it to pre-launch.

8.1 Record what the build cost

In Claude Code, ask for the approximate token count for the work, convert it to Canadian dollars, and append a row to `TOKENS.md`.

📌 TOKEN COUNT

Give me the approximate tokens used from the start of this project to now, then convert those tokens to USD and then to CAD at today's rate. Append the result as a row in `TOKENS.md` with today's date.

Why this is worth the two minutes

The long-term aim is a defensible cost per build, and the input to that is a per-session figure recorded while the session is still in memory. Reconstructing it afterwards is guesswork.

Expected result

TOKENS.md has a row per session and a running total. Operations can see the real cost of a build rather than an estimate.

8.2 Push the branch and open a pull request

Push your branch and open a pull request against main, following HGM-SOP-WEB-002 phase 5.

Expected result

Netlify builds a deploy preview and posts its link on the pull request.

8.3 Post the site for team review

Post the deploy preview link in the **client websites** space in Google Chat and ask the team to review it for mistakes and issues.

 POST IN THE CLIENT WEBSITES SPACE

<client> rebuild is ready for a look: <deploy preview URL>

Original for comparison: <live site URL>

Looking for anything that does not match the original, on desktop and on a phone. Known gaps: <list them>. Comments by <date> please — it moves to pre-launch checks after that.

Expected result

The team has the link and a deadline. Work through what comes back before task 8.4.

Hand the build to pre-launch

Merge the pull request, comment on the Asana task with the `netlify.app` address, the list of keys still missing from task 4.7, and the `TOKENS.md` total, then move the task to the pre-launch stage.

Expected result

The build is finished and somebody else could take it to launch without asking you a question.
Continue in the pre-launch SOP — see section 09.

There is no live site, only a design

This SOP does not apply. A build from a Figma design with no original to copy has a different reference and a different definition of done — see section 09. Do not substitute the design file for the live site here; phases 5 and 7 assume a page you can capture.

The stylesheets are minified or inlined

Common on a built site. Run the saved CSS through a formatter before task 5.6 so the declarations are readable, and where styles are inlined per element, take the values from **Computed** in devtools instead and say so in `VARIABLES.md`.

The client's Supabase project does not exist yet

Do phases 1 to 3, then stop at task 4.2 and ask the account manager to have the client create it. Do not create it under an HGM account to keep moving — it cannot be transferred cleanly later, and the client owns their data.

The template has moved on since this repository was made

Read `UPSTREAM.md` in the repository; it describes how the template's changes are pulled in. Pull them before phase 7, not during it — a template update landing mid-build undoes correction work.

The original site changes during the build

It happens on live businesses. Re-capture only the pages that changed into `Resources/`, note the date in `PLAN.md`, and tell the account manager the change was caught. An uncaptured change is found at client review and reads as a build error.

A page in the original is deliberately not being rebuilt

Name it in `PLAN.md` as out of scope and say what happens to its URL instead — usually a redirect. An unlisted omission is indistinguishable from a page somebody forgot.

SYMPTOM	CAUSE	WHAT TO DO
The first Netlify deploy fails	No environment variables are set yet	Expected until task 4.6. Set the variables, then trigger a new deploy.
Every listing image 404s on localhost	NETLIFY_SITE_ID is set in <code>.env.local</code>	Remove it from <code>.env.local</code> . It flips the site into deployed-Netlify behaviour while you are running locally. Set it on Netlify only.
The Netlify build dies at module resolution	Tracked files import files that were never committed	Run <code>git status</code> , commit everything the changed files import, and push in one commit. A partial push builds a tree that cannot resolve.
The site throws on build with no clear error	NEXT_PUBLIC_SITE_URL is unset	Set it to the <code>netlify.app</code> address, both locally and on Netlify.
Claude ignores the attached capture	The image is wider than about 2000 pixels	Resize the copy you attach so its longest edge is 2000 pixels or less, then attach it again.
Colours are close but not right	Tokens were taken from a capture rather than from the CSS	Rebuild <code>VARIABLES.md</code> from <code>Resources/CSS/</code> per task 5.6, then re-check the build against it.
Corrected sections keep regressing	Looping instead of correcting section by section	Stop looping. Go back to task 7.3 and name each section and each fault in one prompt.
Supabase queries return nothing, with no error	Row level security has no policy for the anon key	Check the policies in the Supabase dashboard. Do not work around it with the service role key in browser code.
The template is missing a file this SOP names	The template has changed	Follow the template's own <code>SETUP.md</code> , finish the build, then tell the Web & Content Specialist so this document is corrected.

Escalate on the template or the build

Leshan Patterson, who owns both template repositories, in the **client websites** space in Google Chat. Use it when the template itself is wrong, stale, or missing something this SOP names.

Escalate on scope or the deadline

Gillian, Operations Manager, in Google Chat. Use it when the work found in the original site is materially larger than the Asana task assumed, before the days are spent rather than after.

💡 Say it early

A build that is going to overrun is visible in phase 5, when you see how many pages and sections the original actually has. That is the cheap moment to raise it. Phase 7 is the expensive one.

08 FINAL CHECKLIST BEFORE YOU HAND THE BUILD ON

- ☐ Every page in the original site's sitemap exists on the build, or is named in `PLAN.md` as out of scope with its redirect.
- ☐ No other client's name appears anywhere in the repository — search the whole project, not just `PLAN.md`.
- ☐ `Resources/` is committed and holds the saved HTML, the page captures, the section captures, the stylesheets and `VARIABLES.md`.
- ☐ Every colour, font and size on the build traces to a value in `VARIABLES.md`, and nothing in that file is still marked unresolved.
- ☐ `.env.local` is not committed, and its current contents are saved in the client's 1Password vault.
- ☐ Every variable on Netlify matches the one in `.env.local`, and the production deploy succeeds.
- ☐ The keys still missing are listed on the Asana task with a named owner for each.
- ☐ A test submission from every form has appeared as a contact in the client's GoHighLevel sub-account.
- ☐ The HGM web team account is a member of the client's Netlify team, and the client is still its Owner.
- ☐ The booking flow runs from the home page search to a property page with dates carried through.
- ☐ Nothing in the pricing or payment path has been altered.
- ☐ The site is on its `netlify.app` address and no DNS record has been changed.
- ☐ `TOKENS.md` has a row for every session and a total.
- ☐ The deploy preview has been posted in the client websites space and the comments have been worked through.

Next: pre-launch readiness checks

[HGM-SOP-WEB-005](#) — SEO and GEO, accessibility to WCAG 2.2 AA, the Lighthouse loop, BrowserStack, legal pages, the pixels, the Stripe and webhook work, the test booking and the DNS cutover. Everything this SOP deliberately left alone.

Then: post-launch connections

[HGM-SOP-WEB-003](#) — Google Analytics 4, Search Console, Hotjar, the structured-data check and the ownership map at handover. Starts the day the site is live.

This SOP is the middle of three. It ends where the site is complete on its temporary address; the next document takes it to launch, and the one after that picks it up once it is live.

DOCUMENT	COVERS	WHEN
HGM-SOP-WEB-004 — this document	Rebuilding a live client site as a Next.js and Supabase site	From the Asana build task to a finished site on <code>netlify.app</code>
HGM-SOP-WEB-005 — pre-launch readiness checks	SEO, GEO, accessibility, Lighthouse, BrowserStack, legal pages, pixels, Stripe and webhooks, the test booking, DNS cutover	Immediately after task 8.4. In draft — ask the Web & Content Specialist for the current version
HGM-SOP-WEB-003 — post-launch connections	GA4, Search Console, Hotjar, structured data, the ownership map	Launch week, after the DNS cutover
HGM-SOP-WEB-002 — Asana to live	Machine setup, branches, pull requests, deploy previews, merging	Referenced from tasks 3.1 and 8.2. Read it first if you have never set this machine up
HGM-SOP-WEB-001 — env files in 1Password	Storing, sharing and updating a client's <code>.env</code> file	Referenced from task 4.7
Building a site from a Figma design	The same build with a design file and no live original as the reference	Not yet written. Raise it with the Web & Content Specialist when a project needs it

💠 The build is not finished when the site looks right

A site that matches the original and has never been through the pre-launch checks is not ready for a client, let alone a domain. Task 8.4 hands it on; do not treat a merged pull request as the end of the job.

10 REVISION HISTORY

DATE	VERSION	AUTHOR	CHANGE
2026-09-22	0.1	Kyle Zinger	First working draft from build notes and screenshots. Launch checks split out to a planned WEB-005.
2026-09-23	0.2	Kyle Zinger	Netlify is the client's account, with HGM added as a Developer (new task 2.1). New tasks 7.4 to 7.6 build the GoHighLevel form and take its embed code.
2026-09-23	0.3	Kyle Zinger	Figure added to task 7.3 — the Claude Code prompt box with an attached capture.
2026-09-23	0.4	Kyle Zinger	Figma removed from the procedure — it belongs to the design-to-site SOP. Design tokens now come from the original site's own stylesheets (tasks 5.5 to 5.7).
2026-10-06	0.5	Kyle Zinger	Twelve image placeholders added at the tasks that still need a capture, ahead of the portal upload.

REMEMBER

"A copy is judged against the original, not against how good it looks on its own."

Next

HGM-SOP-WEB-005 — pre-launch checks